EXPERIMENT NO. 8

APPLICATION OF OPTICAL FLOW FOR REAL-TIME OBJECT TRACKING

AND MOTION ANALYSIS

Name : Arpit Giri

UID : CU24250181

Course : B.Tech CSE

Section : CSE "B"

Roll No. : 11

AIM

To implement an optical flow-based motion analysis system for

real-time object tracking and evaluate its effectiveness in

detecting and tracking moving objects in video sequences

using Python and OpenCV.

Github link :-

https://github.com/cu24250181Ag/Computer_vision_LAB/blob/main/Experiment%207/experiment7.py

STEP 1: IMPORT REQUIRED LIBRARIES

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tkinter import Tk, filedialog
```

STEP 2: UPLOAD VIDEO FROM COMPUTER

```
from google.colab import files

print("Please upload a video file (e.g., .mp4, .avi, .mov).")
uploaded = files.upload()

if not uploaded:
    print("No file uploaded.")
    exit()

video_filename = next(iter(uploaded)) # Get the name of the first uploaded file
print(f"Video file '{video_filename}' uploaded successfully.")
```

```
Please upload a video file (e.g., .mp4, .avi, .mov).
```

Choose FilesScreen Rec...92620.mp4

Screen Recording 2025-11-26 192620.mp4(video/mp4) - 9388171bytes,lastmodified:11/26/2025 - 100%done

```
Saving Screen Recording 2025-11-26 192620.mp4 to Screen Recording 2025-11-26 192620.mp4
Video file 'Screen Recording 2025-11-26 192620.mp4' uploaded successfully.
```

STEP 3: OPEN VIDEO AND READ FIRST FRAME

```
cap = cv2.VideoCapture(video_filename)

if not cap.isOpened():
    raise SystemExit("Unable to open video.")

ret, first_frame = cap.read()

if not ret:
    cap.release()
    raise SystemExit("Unable to read video.")

gray_first = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
```

STEP 4: SHI-TOMASI CORNER DETECTION

```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    gray_first,
    mask=None,
```

```
    **feature_params
)

if p0 is None:
    cap.release()
    raise SystemExit("No feature points detected.")

# Create a copy for displaying detected points
shi_tomasi_frame = first_frame.copy()

for point in p0:
    x, y = point.ravel().astype(int)
    cv2.circle(shi_tomasi_frame, (x, y), 4, (0, 255, 0), -1)
```

STEP 5: LUCAS-KANADE OPTICAL FLOW TRACKING

```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)


old_gray = gray_first.copy()
old_points = p0.copy()

trajectory_frame = first_frame.copy()

previous_center = np.mean(old_points.reshape(-1, 2), axis=0)
total_displacement = 0
frame_count = 0

fps = cap.get(cv2.CAP_PROP_FPS)

if fps <= 0:
    fps = 30
```

STEP 6: TRACK OBJECT AND DRAW MOTION TRAJECTORY

```
while True:

    ret, frame = cap.read()

    if not ret:
        break

    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    new_points, status, error = cv2.calcOpticalFlowPyrLK(
        old_gray,
        gray_frame,
        old_points,
        None,
        **lk_params
    )

    if new_points is None:
        break

    good_new = new_points[status == 1]
    good_old = old_points[status == 1]

    if len(good_new) == 0:
        break

    # Calculate object center
    current_center = np.mean(good_new, axis=0)

    # Calculate displacement
    displacement = np.linalg.norm(
        current_center - previous_center
    )

    total_displacement += displacement
    frame_count += 1

    # Draw motion vectors
    for new, old in zip(good_new, good_old):
```

```
        x_new,    y_new=new.ravel().astype(int)
        x_old, y_old=old.ravel().astype(int)

        cv2.arrowedLine(
            trajectory_frame,
            (x_old,   y_old),
            (x_new,   y_new),
            (0, 255, 0),
            2,
            tipLength=0.3
        )

        cv2.circle(
            trajectory_frame,
            (x_new, y_new),
            3,
            (0, 0, 255),
            -1
        )

    # Draw center point
    center_x, center_y=current_center.astype(int)

    cv2.circle(
        trajectory_frame,
        (center_x, center_y),
        6,
        (255, 0, 0),
        -1
    )

    previous_center = current_center

    old_gray=gray_frame.copy()
    old_points=good_new.reshape(-1, 1, 2)
```

STEP 7: CALCULATE MOTION DIRECTION AND AVERAGE SPEED

```
if frame_count > 0:

    average_displacement = total_displacement / frame_count

    average_speed = average_displacement * fps

    dx = current_center[0] - previous_center[0]
    dy = current_center[1] - previous_center[1]

    if abs(dx) > abs(dy):

        if dx > 0:
            direction = "Right"
        else:
            direction = "Left"

    else:

        if dy > 0:
            direction = "Down"
        else:
            direction = "Up"

else:

    average_displacement = 0
    average_speed = 0
    direction = "No Motion"
```

STEP 8: FARNEBACK DENSE OPTICAL FLOW

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret, frame2 = cap.read()

if not ret:
    cap.release()
    raise SystemExit("Unable to read consecutive frames.")

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
```

```
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)
```

STEP 9: CALCULATE MAGNITUDE AND DIRECTION

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

average_flow_magnitude = np.mean(magnitude)
```

STEP 10: CREATE COLOR-CODED DENSE OPTICAL FLOW

```
hsv = np.zeros_like(frame1)

hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

dense_flow_frame = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)
```

STEP 11: DISPLAY FOUR MAIN OUTPUTS

```
plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(
    shi_tomasi_frame,
    cv2.COLOR_BGR2RGB
))
plt.title("Output 1: Shi-Tomasi Feature Points")
plt.axis("off")
plt.show()


plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(
    trajectory_frame,
    cv2.COLOR_BGR2RGB
))
plt.title("Output 2: Lucas-Kanade Tracking and Trajectory")
plt.axis("off")
plt.show()

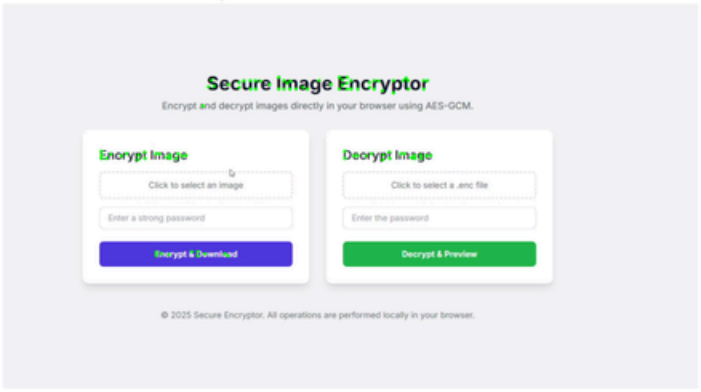

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(
    frame2,
    cv2.COLOR_BGR2RGB
))
plt.title("Output 3: Consecutive Video Frame")
plt.axis("off")
plt.show()


plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(
    dense_flow_frame,
    cv2.COLOR_BGR2RGB
```

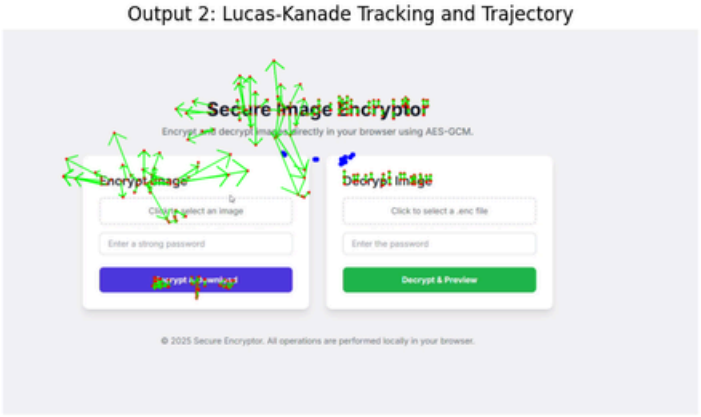
```
))
plt.title("Output 4: Farneback Dense Optical Flow")
plt.axis("off")
plt.show()


cap.release()
```

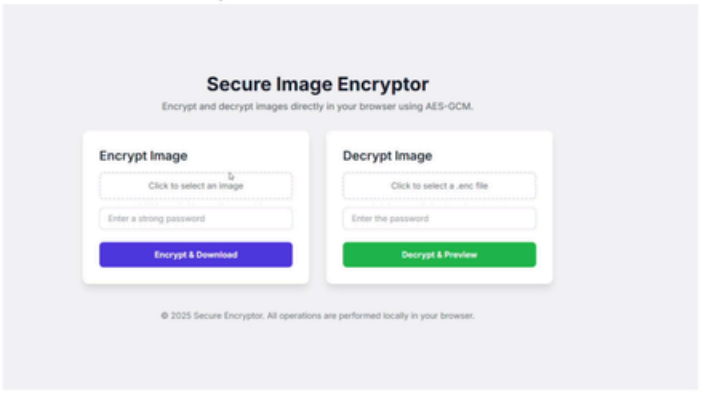
Output 1: Shi-Tomasi Feature Points



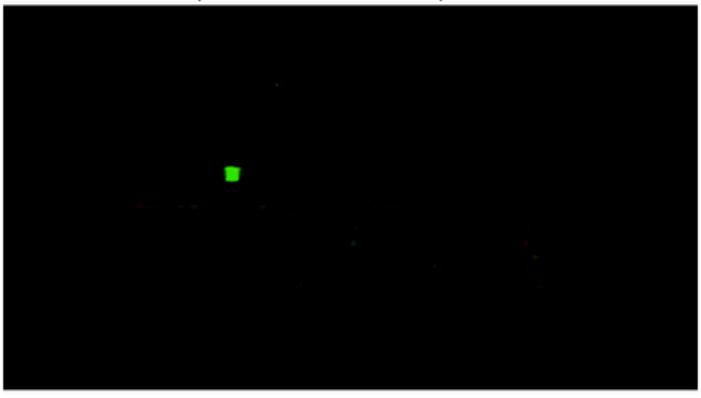
Output 2: Lucas-Kanade Tracking and Trajectory



Output 3: Consecutive Video Frame



Output 4: Farneback Dense Optical Flow



QUESTIONS AND ANSWERS

Q1. How does object tracking differ from object detection?

Answer:
Object detection identifies objects in individual frames, whereas object tracking follows the movement of an already identified object across consecutive video frames.

Q2. Explain how Optical Flow can be used for real-time object tracking.

Answer:
Optical Flow estimates the movement of pixels or feature points between consecutive frames. These motion vectors can be used to continuously track the position and movement of an object.

Q3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Answer:
Shi-Tomasi Corner Detection selects strong and reliable feature points from the image. Lucas-Kanade then tracks these feature points between consecutive frames.

Q4. Why is Optical Flow suitable for motion analysis in videos?

Answer:
Optical Flow provides information about the direction and magnitude of motion between video frames. Therefore, it can be used to analyze object movement and motion patterns.

Q5. What challenges arise while tracking fast-moving or partially occluded objects?

Answer:
Fast-moving objects may move significantly between frames, making feature tracking difficult. Partial occlusion can hide important feature points and may cause tracking errors.

Q6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Answer:
Optical Flow mainly uses pixel or feature-point motion and can work with relatively low computational requirements. Deep learning-based tracking can provide stronger object recognition and tracking capabilities but generally requires more computational resources and trained models.

Q7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Answer:
Optical Flow can estimate the movement of vehicles, pedestrians, and other objects. This motion information can help in traffic monitoring, obstacle detection, navigation, and understanding the movement of surrounding objects.

Q8. Explain the effect of camera motion on Optical Flow estimation.

Answer:
Camera movement causes many or most pixels in the scene to move simultaneously. This can create large optical-flow vectors even when objects themselves are stationary, making object motion analysis more difficult.

Q9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

Answer:

1. Video surveillance
2. Traffic monitoring
3. Autonomous driving
4. Human activity recognition
5. Sports analytics

Q10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

Answer:
Optical Flow provides motion information that helps systems identify moving objects, estimate their direction and speed, follow their trajectories, and recognize movement patterns.

RESULT

The optical flow-based object tracking and motion analysis system was successfully implemented using Python and OpenCV. Shi-Tomasi Corner Detection, Lucas-Kanade Sparse Optical Flow, and Farneback Dense Optical Flow were used to analyze object movement, trajectory, direction, displacement, and motion patterns in video sequences.